

Pembangunan SIG Web

Dari Library ke Sistem: Membangun WebGIS Nyata

Program Sarjana Terapan Teknologi Survei dan Pemetaan Dasar
Departemen Teknologi Kebumian
Sekolah Vokasi, UGM

Ismail Sunni | Geospatial Software Engineer | Camptocamp DE

Presentasi Ini

<https://github.com/ismailsunni/web-gis-2026>



Recap: Apa yang Sudah Kalian Tahu

Minggu 4:

- JavaScript: variabel, fungsi, objek, OOP
- DOM manipulation & event listener
- Mini WebGIS dari nol (tanpa library)

Minggu 5:

- Pengenalan Leaflet & OpenLayers sebagai library
- Client-side scripting dengan library GIS

Hari ini: Naik level — dari *mengenai library* ke *membangun sistem*

Tujuan Pembelajaran

Setelah sesi ini, mahasiswa mampu:

- ✓ Membandingkan **WebGIS vs Desktop GIS** secara mendalam
- ✓ Memilih **Maps API yang tepat** untuk kasus nyata
- ✓ Mengintegrasikan **layer WMS/WFS** dari server OGC
- ✓ Memuat data dari **sumber eksternal** (Google Sheets, GeoJSON URL)
- ✓ Men-deploy WebGIS ke **GitHub Pages** secara langsung

Outline

1. **WebGIS vs Desktop GIS** — Perbandingan mendalam
2. **Arsitektur WebGIS** — Stack & komponen
3. **Pilih Maps API yang Tepat** — Framework keputusan
4. **Hands-on: Upgrade WebGIS** — WMS, data eksternal, filter
5. **Deploy Live** — GitHub Pages step-by-step
6. **Kenalan: MapLibre GL JS** — Selanjutnya
7. **Studi Kasus** — WebGIS di dunia nyata
8. **Refleksi & Tugas**

1 WebGIS vs Desktop GIS

Lebih dari Sekadar "Bisa Online"

Desktop GIS: Kekuatan Utama

QGIS / ArcGIS Desktop / GRASS

- Analisis spasial lengkap — buffer, overlay, network analysis
- Editing geometri presisi tinggi
- Cartographic production — peta cetak berkualitas
- Processing pipeline — batch geoprocessing
- Data format beragam — Shapefile, GeoPackage, GDB, raster

Desktop GIS = laboratorium analisis spasial

Web GIS: Kekuatan Utama

Google Maps / Mapbox / Custom WebGIS

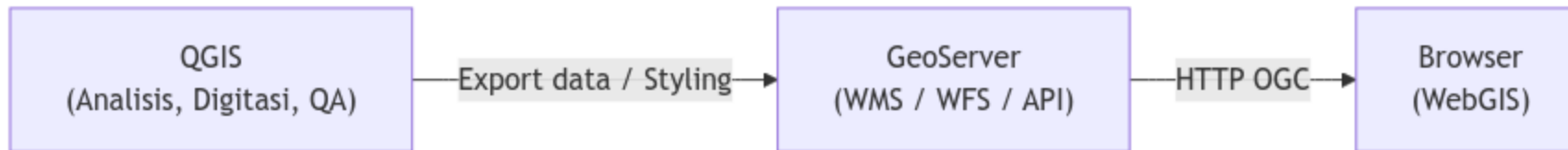
- **Zero install** — akses via URL, tidak perlu instalasi software
- **Multi-platform** — desktop, tablet, smartphone (responsive)
- **Kolaborasi real-time** — banyak user akses bersamaan tanpa konflik
- **Integrasi seamless** — embed di website, dashboard, CMS, aplikasi
- **Update sentral** — data berubah satu kali, semua user dapat update
- **Aksesibilitas** — publik bisa akses tanpa login atau izin khusus
- **Mobile-first** — optimized untuk penggunaan di lapangan via smartphone
- **Lightweight** — tidak butuh hardware canggih untuk client

Web GIS = media komunikasi, distribusi, dan publikasi spasial

Perbandingan Mendalam

Aspek	Desktop GIS	Web GIS
Analisis	Lengkap (500+ tools)	Terbatas pada visualisasi & query
Data size	GB–TB lokal	MB–ratusan MB streaming
CRS	Semua CRS	Umumnya Web Mercator (EPSG:3857)
Rendering	GPU lokal	Browser (Canvas/WebGL)
Offline	✅ Penuh	⚠️ Membutuhkan caching
Kolaborasi	❌ File-based	✅ Real-time multi-user
Distribusi	Email/FTP file	Cukup share URL
Maintenance	Update per mesin	Update 1x di server (otomatis)

Hybrid Approach: Best of Both Worlds



Workflow modern: Analisis di desktop → Publikasi di web

Kapan Pakai Yang Mana?



Desktop GIS:

- Analisis kompleks (interpolasi, network, 3D)
- Digitasi presisi
- Peta cetak / kartografi



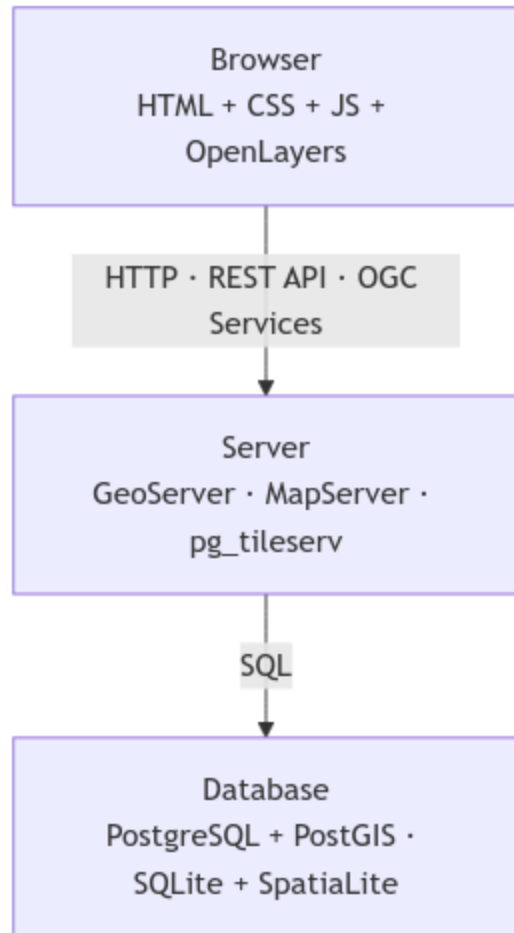
Web GIS:

- Dashboard monitoring
- Informasi publik
- Kolaborasi tim
- Aplikasi mobile/field

2 **Arsitektur WebGIS**

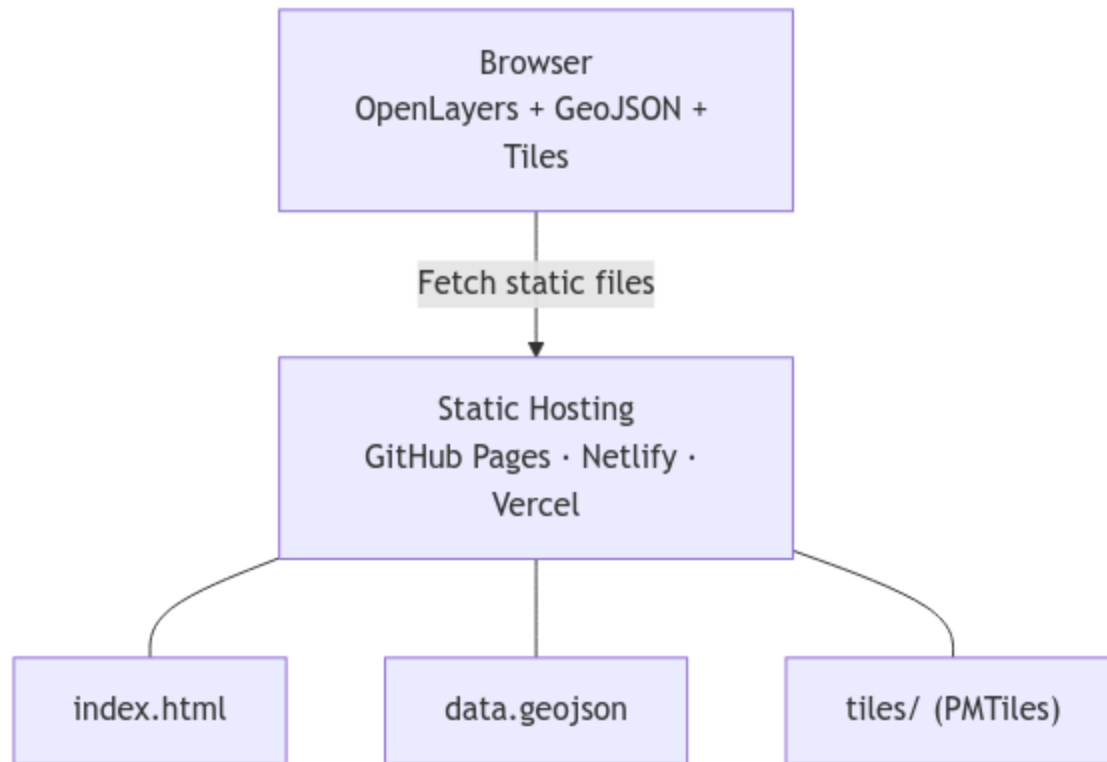
Dari Database ke Browser

Full Stack WebGIS



Alternatif: Serverless WebGIS

Tidak semua WebGIS butuh backend!



Gratis, cepat, tanpa server!

Format Data di WebGIS

Format	Tipe	Cocok untuk
GeoJSON	Vektor	Fitur kecil-sedang (<10 MB)
TopoJSON	Vektor (compressed)	Fitur besar, batas wilayah
XYZ Tiles	Raster	Base map (OSM, satellite)
PMTiles	Vektor tiles	Offline / static hosting
WMS	Raster	Server-rendered map image
WFS	Vektor	Server feature access
COG	Raster (cloud)	Citra satelit besar

3 Ragam Maps API

Perbandingan & Pemilihan

Maps API #1: Leaflet.js

"Ringan & Cepat"

 Leaflet.js adalah library JavaScript terkecil dan paling mudah dipelajari.

Karakteristik:

- Ukuran — 42 KB gzip (terKecil!)
- Syntax — Sederhana & intuitif
- Plugin — Ecosystem besar & matang
- Support — Komunitas aktif

Cocok untuk:

-  WebGIS sederhana di website
-  Dashboard monitoring

Maps API #2: Google Maps API

"Siap Pakai & Powerful"

📍 **Google Maps API** adalah layanan cloud siap pakai dari Google dengan kapabilitas luas.

Karakteristik:

- **Hosting** — Cloud service (hosted oleh Google)
- **API** — Sangat generous & well-documented
- **Basemap** — Berkualitas tinggi & up-to-date
- **Features** — Street View, Directions, Places, dsb

Cocok untuk:

-  Aplikasi commercial/berbayar

Maps API #3: OpenLayers

"Professional & Standard"

 OpenLayers adalah library enterprise-grade dengan standar OGC native.

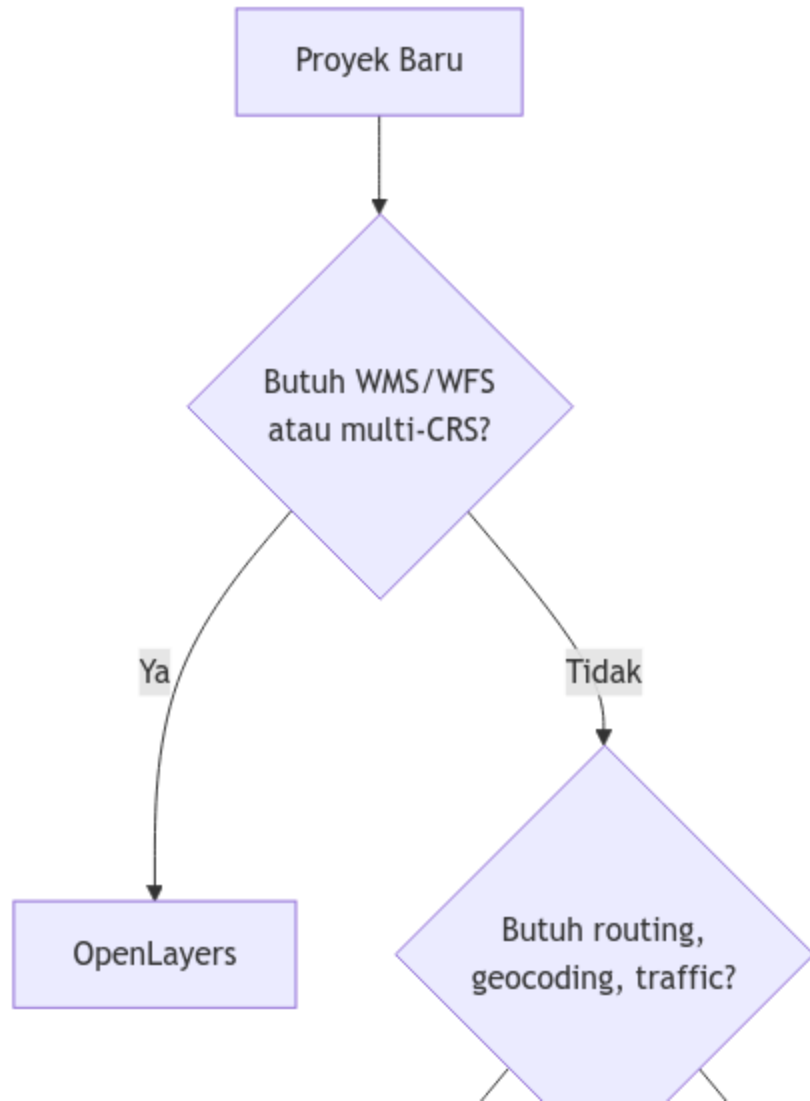
Karakteristik:

- **Ukuran** — 750+ KB gzip (terbesar, tapi powerful)
- **Standard** — OGC-native (WMS, WFS, GML)
- **Projection** — Support multi-CRS & custom projection
- **Features** — Map rotation, 3D-ready, layer management

Cocok untuk:

-  WebGIS profesional
-  Geoportal nasional/regional

Framework Keputusan



Leaflet vs Google Maps vs OpenLayers: Detail

Kriteria	Leaflet	Google Maps	OpenLayers
Setup	Import CDN	Dapatkan API key	Import CDN
Dokumentasi	Lengkap	Sangat lengkap, examples banyak	Lengkap, technical
Learning curve	Mudah (2–4 jam)	Mudah (2–4 jam)	Menengah (6–10 jam)
Basemap	Plugin/tertiary	Built-in, premium	Open (OSM, CARTO, Esri)
Customization	Baik	Terbatas (design constraints)	Sangat baik
WMS/WFS	Plugin/sulit	✗ Tidak native	✓ Built-in, native

Google Maps: Kapan Pakai?

✓ Gunakan jika:

- Aplikasi **commercial/berbayar**
- Target **non-GIS user** (consumer)
- Perlu **Street View** atau **Directions API**
- Budget ada untuk API cost

✗ Hindari jika:

- Data proyeksi custom (non-Web Mercator)
- Butuh WMS/WFS dari server pemetaan lokal
- Budget terbatas, traffic tinggi
- Perlu open source untuk compliance

Contoh: Google Maps API

```
const map = new google.maps.Map(document.getElementById('map'), {  
  zoom: 14,  
  center: { lat: -7.7956, lng: 110.3695 }  
});  
  
const marker = new google.maps.Marker({  
  position: { lat: -7.7956, lng: 110.3695 },  
  map: map,  
  title: 'Yogyakarta'  
});
```

Keuntungan: Instant, polished, banyak plugin

Kerugian: API key required, metered billing

Leaflet vs OpenLayers: Teknis

Aksi	Leaflet	OpenLayers
Buat peta	<code>L.map('map')</code>	<code>new ol.Map({target:'map'})</code>
Set view	<code>.setView([-7.79, 110.36], 13)</code>	<code>view: new ol.View({center, zoom})</code>
Tambah tile	<code>L.tileLayer(url).addTo(map)</code>	<code>new ol.layer.Tile({source:...})</code>
Marker	<code>L.marker([lat, lng])</code>	<code>new ol.Feature({geometry: Point})</code>
Popup	<code>.bindPopup(html)</code>	<code>new ol.Overlay({element:...})</code>
WMS	Plugin (sulit)	 <code>ol.source.TileWMS</code> (mudah)

4 Hands-on: Praktik WebGIS dengan Strategi Publikasi

Dari Local → Public dengan Akses Terkontrol

Pilihan Skenario Praktik

Opsi A: Public Dashboard (GitHub Pages)

Requirements:

- OpenLayers + data dari Google Sheets
- Basemap switcher (OSM + CARTO)
- Filter kategori interaktif
- Deploy ke GitHub Pages

Keuntungan: Gratis, instant, langsung online

Tantangan: No backend, data hardcoded/sheet URL saja

Opsi B: Protected Dashboard (Vercel + Auth)

Upgrade 1: WMS Layer dari Server OGC

OpenLayers bisa langsung bicara dengan GeoServer, QGIS Server, BIG, dsb.

```
const wmsLayer = new ol.layer.Tile({
  source: new ol.source.TileWMS({
    url: 'https://geoserver.example.com/wfs',
    params: {
      'LAYERS': 'nama:layer',
      'TILED': true,
      'FORMAT': 'image/png',
      'TRANSPARENT': true
    },
    serverType: 'geoserver'
  }),
  opacity: 0.7
});
map.addLayer(wmsLayer);
```

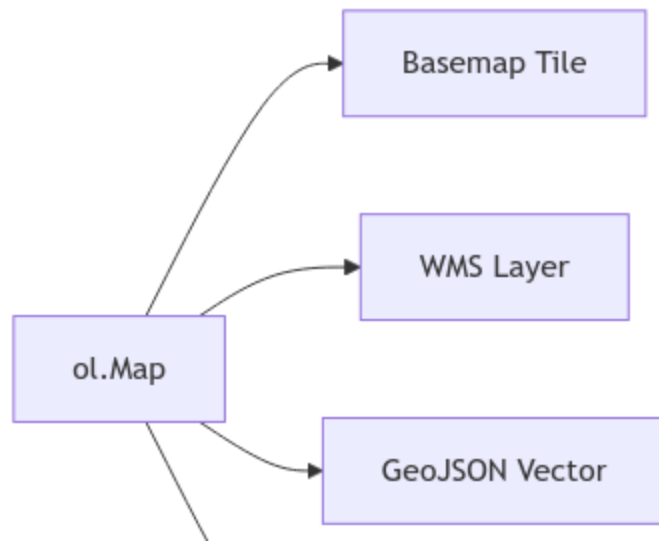


Coba dengan WMS publik: BIG (ina-geoportal.go.id), BMKG, atau

Toggle Visibility Layer

```
// Tombol toggle di HTML  
// <button id="toggle-wms">Toggle WMS</button>  
  
document.getElementById('toggle-wms').addEventListener('click', () => {  
    wmsLayer.setVisible(!wmsLayer.getVisible());  
});
```

Struktur layer dalam OpenLayers:



Upgrade 2: Data dari Google Sheets

Konsep: Google Sheet publik → CSV → parse di JavaScript → OL Features

Google Sheet (isi data) → Publish as CSV → Fetch di JS → Buat Feature OL

Setup Google Sheet:

nama	lon	lat	kategori	deskripsi
Tugu Yogya	110.3672	-7.7828	landmark	Ikon kota
UGM	110.3780	-7.7703	universitas	Kampus tertua

Fetch & Parse Google Sheets CSV

```
const SHEET_CSV_URL =  
  'https://docs.google.com/spreadsheets/d/ID/export?format=csv&gid=0';  
  
async function loadFromSheet(url) {  
  const res = await fetch(url);  
  const text = await res.text();  
  const rows = text.split('\n').slice(1); // skip header  
  
  rows.forEach(row => {  
    const [nama, lon, lat, kategori, deskripsi] = row.split(',');  
    if (!lon || !lat) return;  
  
    const feature = new ol.Feature({  
      geometry: new ol.geom.Point(  
        ol.proj.fromLonLat([parseFloat(lon), parseFloat(lat)])  
      ),  
      nama, kategori, deskripsi  
    });  
    markerSource.addFeature(feature);  
  });  
}  
  
loadFromSheet(SHEET_CSV_URL);
```

Upgrade 3: Filter Kategori

```
<!-- Tambah di HTML -->
<div id="filter">
  <button data-cat="all">Semua</button>
  <button data-cat="landmark">Landmark</button>
  <button data-cat="universitas">Universitas</button>
</div>
```

```
document.querySelectorAll('#filter button').forEach(btn => {
  btn.addEventListener('click', () => {
    const cat = btn.dataset.cat;
    markerLayer.setStyle(feature => {
      if (cat === 'all' || feature.get('kategori') === cat) {
        return circleStyle(feature); // tampilkan
      }
      return null; // sembunyikan
    });
  });
});
```

5 Strategi Publikasi & Akses Peta Interaktif

Platform Publikasi WebGIS

GitHub Pages (Gratis, serverless)

- Untuk: Proyek statis, demo, portfolio
- Setup: Push ke GitHub → auto-deploy
- **Pro:** Gratis, custom domain, HTTPS, git-managed
- **Kontra:** Static only, tidak ada backend

Vercel / Netlify (Gratis + upgrade)

- Untuk: WebGIS dengan serverless functions
- Setup: Connect GitHub → auto-deploy on push
- **Pro:** Lebih fast, analytics, preview URLs
- **Kontra:** Limited free tier

Skenario Publikasi

Skenario 1: WebGIS Sederhana (Peta Lokasi Toko)

→ Leaflet + GeoJSON lokal → GitHub Pages

```
<!-- Hanya HTML + JS, data hardcoded atau GeoJSON file -->  
<!-- Deploy: git push → online dalam 1 menit -->
```

Skenario 2: Dashboard Monitor Data Dinamis

→ OpenLayers + REST API (external) → Vercel / DigitalOcean

```
// Fetch data dari API endpoint setiap 5 detik  
setInterval(() => fetch('/api/features').then(...), 5000);
```

Skenario 3: Geoportal Nasional / Provinsi

→ GeoServer + PostGIS + OpenLayers + Portal → Server dedicated

Strategi Akses Peta Interaktif: 4 Level

Level 1: Public (No Auth)

- Siapa saja bisa akses via URL
- Data publik (batas wilayah, POI umum)
- Contoh: map.geo.admin.ch, OpenStreetMap

Level 2: Authenticated (Login)

- Google/SSO login, atau custom auth
- Data sensitive tapi untuk tim/organisasi
- Backend tracking: siapa akses kapan
- Contoh: internal company GIS dashboard

Implementasi: Public WebGIS di GitHub Pages

```
# 1. Init repo lokal
git init
git add index.html app.js data/landmarks.geojson
git commit -m "WebGIS Yogyakarta - initial"

# 2. Buat repo di GitHub, lalu push
git remote add origin https://github.com/USERNAME/webgis-yogya.git
git push -u origin main

# 3. Aktifkan GitHub Pages
# Settings → Pages → Branch: main → Folder: root → Save

# 4. Tunggu ~1 menit, akses:
# https://USERNAME.github.io/webgis-yogya/
```

Implementasi: Autentikasi + Backend (Vercel)

```
// API endpoint: /api/landmarks?token=XXX
// Backend mengecek token sebelum return data

const token = localStorage.getItem('auth_token');
fetch(`/api/landmarks?token=${token}`)
  .then(r => r.json())
  .then(data => {
    data.features.forEach(f => {
      new L.Marker([f.geometry.coordinates[1], f.geometry.coordinates[0]])
        .bindPopup(f.properties.nama)
        .addTo(map);
    });
  });
```

Best Practices: Tips Publikasi Profesional

Brand & UX

1. **README.md** — sertakan screenshot + link demo + citation
2. **Responsive design** — test mobile (DevTools: toggle device)
3. **Loading feedback** — spinner/progress when fetching data
4. **Error handling** — graceful fallback jika API down

Technical

1. **CORS headers** — pastikan data eksternal allow cross-origin
2. **HTTPS everywhere** — GitHub Pages otomatis; WMS server juga harus HTTPS
3. **Performance** — compress GeoJSON, pakai CDN, lazy-load tiles
4. **Caching** — set proper Cache-Control headers

Contoh Publikasi Nyata

Sheet Map (Leaflet + Google Sheets + GitHub Pages)

ismailsunni.id/map/sheet-map

- Data dari spreadsheet publik
- Clustering otomatis
- 100% client-side, zero backend
- Deploy strategy: manual push via git

Route Finder (OpenLayers + Backend API)

ismailsunni.id/map/route-finder

- Backend: Node.js + PostgreSQL
- Hosted di DigitalOcean

7 Kenalan: MapLibre GL JS

Selanjutnya Setelah OpenLayers

Mengapa MapLibre GL JS?

OpenLayers dan Leaflet render peta sebagai **raster (Canvas)**.
MapLibre GL JS render dengan **WebGL** — beda kategori.

	OpenLayers / Leaflet	MapLibre GL JS
Render	Canvas 2D	WebGL
Tiles	Raster (PNG)	Vector tiles (MVT)
3D	✗	✓ Terrain, buildings
Rotasi peta	Terbatas	✓ Bebas
Animasi	Terbatas	✓ Smooth
Style	JS	JSON style spec

MapLibre: Contoh Singkat

```
const map = new maplibregl.Map({  
  container: 'map',  
  style: 'https://tiles.openfreemap.org/styles/liberty',  
  center: [110.3695, -7.7956],  
  zoom: 14,  
  pitch: 45,          // tampilan miring (3D effect)  
  bearing: -20        // rotasi peta  
});  
  
map.addControl(new maplibregl.NavigationControl());
```

Kapan pakai MapLibre: visualisasi 3D, animasi data, vektor tiles skala besar

 maplibre.org — open source, gratis, aktif dikembangkan

8 Studi Kasus

WebGIS di Dunia Nyata

Studi Kasus: map.geo.admin.ch

 **Geoportal nasional Swiss** — dibangun dengan OpenLayers

Fitur:

- 800+ layer data nasional
- 2D dan 3D view
- WMS/WMTS dari swisstopo
- Pencarian alamat & koordinat
- Cetak peta & share URL
- Open source!

 map.geo.admin.ch

Studi Kasus: Peta Interaktif Routing

 **Route Finder** — contoh WebGIS dengan routing nyata

Fitur:

- Pencarian lokasi (Photon geocoding)
- Routing jalan nyata via pgRouting
- TSP solver (Held-Karp)
- Multiple kota (Yogyakarta & München)

 ismailsunni.id/map/route-finder

Dibangun dengan OpenLayers + Supabase + PostgreSQL

Studi Kasus: Peta dari Google Sheet

 **Sheet Map** — WebGIS tanpa backend, data dari spreadsheet

Fitur:

- Data dari Google Sheet publik
- Clustering otomatis
- Warna per kategori
- Zero server — 100% client-side

 ismailsunni.id/map/sheet-map

Cocok untuk: survei lapangan, data crowdsource, tugas mahasiswa

9 Refleksi & Tugas

Refleksi

- 🤔 Kapan sebaiknya pakai Desktop vs Web GIS?
- 🤔 Kapan WMS lebih baik daripada GeoJSON langsung?
- 🤔 Apa keuntungan load data dari Google Sheets vs hardcode?
- 🤔 Mengapa MapLibre berbeda kategori dari Leaflet/OpenLayers?

Key Takeaways

1. Desktop GIS = **analisis**, Web GIS = **komunikasi & distribusi**
2. OpenLayers unggul di **OGC standards** — WMS/WFS, multi-CRS, native
3. **Google Sheets** → **CSV** = cara cepat punya backend data tanpa server
4. **Serverless WebGIS** bisa sangat powerful (GeoJSON + GitHub Pages)
5. MapLibre GL JS = WebGL rendering, vector tiles, 3D — generasi berikutnya

Tugas Publikasi & Akses

Wajib (Poin A: GitHub Pages)

Buat **WebGIS** interaktif public dengan ketentuan:

1. Pilih maps API: **Leaflet ATAU OpenLayers**
2. Data source: **Google Sheets** (bukan hardcode) — minimal 10 lokasi
3. Base maps: **minimal 2 pilihan** + switcher UI
4. Filtering: **minimal 2 kategori** dengan interaktif toggle
5. UI Elements:
 - Header/title + attributions
 - Legend (warna per kategori)
 - Loading state (spinner)

Referensi

- [OpenLayers Quick Start](#)
- [OpenLayers Examples](#)
- [MapLibre GL JS Docs](#)
- [GitHub Pages Docs](#)
- [GeoJSON.io](#) — buat GeoJSON interaktif
- [ina-geoportal.go.id](#) — WMS BIG
- [Map Collection](#) — contoh proyek nyata
- [This presentation](#)

Eksplorasi dan bawa pertanyaan ke sesi berikutnya!